

58. Fetch: Cross-Origin Requests (CORS)

Dusri website par `fetch` bhejoge to aksar **fail** hoga:

```
js

try { await fetch('http://example.com'); }
catch (err) { alert(err); } // Failed to fetch
```

Origin = domain + port + protocol. Dusre domain (subdomain bhi), protocol ya port par jaane wali requests ko **cross-origin** kehte hain, aur unhe remote side se special headers chahiye. Is policy ko **CORS** (Cross-Origin Resource Sharing) kehte hain.

CORS kyun? (thoda itihaas)

Internet ko evil hackers se bachane ke liye. Pehle ek site ki script dusri site ka content access nahi kar sakti thi (jaise `hacker.com` `gmail.com` ka mailbox nahi padh sakta).

Developers ne workarounds nikaale:

- **Forms:** dusre server par `<form>` submit karna (iframe me). Par response padh nahi sakte the.
- **Scripts (JSONP):** `<script src="http://another.com/weather.json?callback=gotWeather">`. Server aisi script deta jo `gotWeather({...})` call karti. Dono taraf ki sahmati se hota tha, isliye theek. Purane browsers me ab bhi chalta hai.

Baad me networking methods aaye, cross-origin allow hue, par **naye capabilities ke liye server ki explicit permission** (special headers) chahiye.

Safe requests

Do type ki cross-origin requests: **safe** aur baaki sab.

Safe request ki 2 shartein:

1. Method: `GET`, `POST` ya `HEAD`
2. Headers: sirf ye custom headers allowed:
 - `Accept`, `Accept-Language`, `Content-Language`
 - `Content-Type` = `application/x-www-form-urlencoded`, `multipart/form-data` ya `text/plain`

Baaki sab **unsafe**. Farak ye ki safe requests `<form>` ya `<script>` se purane zamane me bhi ho sakti thi, isliye purana server bhi unke liye taiyar hota hai. `DELETE` ya `API-Key` header wali requests pehle possible nahi thi, isliye purane servers maan sakte hain ki "aisi request browser page se aa hi nahi sakti (privileged source hai)".

Safe requests ke liye CORS

Cross-origin request me browser hamesha `Origin` header jodta hai (sirf origin, path nahi):

```
GET /request
Host: anywhere.com
Origin: https://javascript.info
```

Server manzoor kare to response me `Access-Control-Allow-Origin` daalta hai (hamara origin ya `*`). Warna error.

Browser beech ka bharosemand vichaulia hai:

1. Sahi `Origin` bhejta hai.
2. Response me `Access-Control-Allow-Origin` check karta hai. Ho to JS ko response dikhata hai, warna error.

Response headers

Default me JS sirf ye "safe" response headers padh sakta hai: `Cache-Control`, `Content-Language`, `Content-Length`, `Content-Type`, `Expires`, `Last-Modified`, `Pragma`.

Aur headers padhne ke liye server `Access-Control-Expose-Headers` me unke naam (comma se) bhejta hai:

```
Access-Control-Allow-Origin: https://javascript.info
Access-Control-Expose-Headers: Content-Encoding,API-Key
```

Unsafe requests: preflight

Kisi bhi HTTP method (`PATCH`, `DELETE`...) ki request ho sakti hai, par unsafe request browser **seedhe nahi bhejta**. Pehle ek **preflight** request bhejta hai permission maangne ke liye.

Preflight: method `OPTIONS`, body nahi, 3 headers:

- `Access-Control-Request-Method`: asli request ka method
- `Access-Control-Request-Headers`: unsafe headers ki list
- `Origin`

Server 200 status aur khaali body ke saath jawab de:

- `Access-Control-Allow-Origin`: `*` ya hamara origin
- `Access-Control-Allow-Methods`: allowed methods
- `Access-Control-Allow-Headers`: allowed headers
- (optional) `Access-Control-Max-Age`: permission kitne seconds cache ho (tab tak dobara preflight nahi)

Example: cross-origin `PATCH`

```
js  
  
fetch('https://site.com/service.json', {  
  method: 'PATCH',  
  headers: { 'Content-Type': 'application/json', 'API-Key': 'secret' }  
});
```

Ye unsafe kyun: method `PATCH`, `Content-Type` `application/json` (safe list me nahi), aur `API-Key` header. (Ek karan hi kaafi hai.)

Step 1 (preflight request):

```
OPTIONS /service.json  
Origin: https://javascript.info  
Access-Control-Request-Method: PATCH  
Access-Control-Request-Headers: Content-Type,API-Key
```

Step 2 (preflight response):

```
200 OK  
Access-Control-Allow-Origin: https://javascript.info  
Access-Control-Allow-Methods: PUT,PATCH,DELETE  
Access-Control-Allow-Headers: API-Key,Content-Type,If-Modified-Since,Cache-Control  
Access-Control-Max-Age: 86400
```

Step 3 (asli request) jaati hai (safe wali scheme jaisi, `Origin` ke saath). **Step 4 (asli response)** me server ko phir se `Access-Control-Allow-Origin` dena **zaruri** hai. Preflight success hone se ye chhoot nahi milti.

Preflight "parde ke peeche" hota hai, JS ko dikhta nahi. JS ko sirf main request ka response ya error milta hai.

Credentials

Cross-origin request **by default cookies ya HTTP authentication nahi bhejti** (`another.com` ki apni cookies bhi nahi). Kyunki credentials wali request bahut powerful hoti hai (user ki taraf se kaam karna, sensitive info).

Bhejne ke liye:

```
js
```

```
fetch('http://another.com', { credentials: "include" });
```

Server ko bhi (`Access-Control-Allow-Credentials: true`) dena hoga (`Access-Control-Allow-Origin` ke saath):

```
Access-Control-Allow-Origin: https://javascript.info  
Access-Control-Allow-Credentials: true
```

Credentials ke saath (`Access-Control-Allow-Origin`) me (*) **allowed nahi**, exact origin dena padega.

Summary

Safe requests: browser (`Origin`) bhejta hai. Server (`Access-Control-Allow-Origin`) (aur credentials ho to (`Access-Control-Allow-Credentials: true`)) deta hai. Extra response headers ke liye (`Access-Control-Expose-Headers`).

Unsafe requests: pehle (`OPTIONS`) preflight (`Access-Control-Request-Method`, `Access-Control-Request-Headers`), server (`Access-Control-Allow-Methods`, `Access-Control-Allow-Headers`, `Access-Control-Max-Age`) se jawab de, phir asli request.

Task: (`Origin`) kyun chahiye jab (`Referer`) hai?

(`Referer`) kabhi kabhi **absent** hota hai (jaise HTTPS page se HTTP fetch), Content Security Policy ise rok sakti hai, aur spec me ye optional hai, aur (`fetch`) me ise hata/badal bhi sakte hain. Isliye bharosemand (`Origin`) banaya gaya: cross-origin requests ke liye browser sahi (`Origin`) ki guarantee deta hai.